

TomaLayer: the Shared Building Block of TomaNet

What it does, what it takes, and how the same block builds both models

Tomato disease detection project

September 17, 2026

1 The idea in one paragraph

TomaNet is built from one small unit, **TomaBlock**. Stack n of those back to back and you have a **TomaLayer**. That is the whole abstraction: **TomaLayer** is not a new kind of layer, it is a repeat-counter around **TomaBlock**. The interesting part is where it gets used — the same **TomaLayer** code builds the four backbone stages that TOMANET-C and TOMANET-D *share*, and the four extra fusion points that only TOMANET-D has. Nothing is re-implemented per task; only the arguments change.

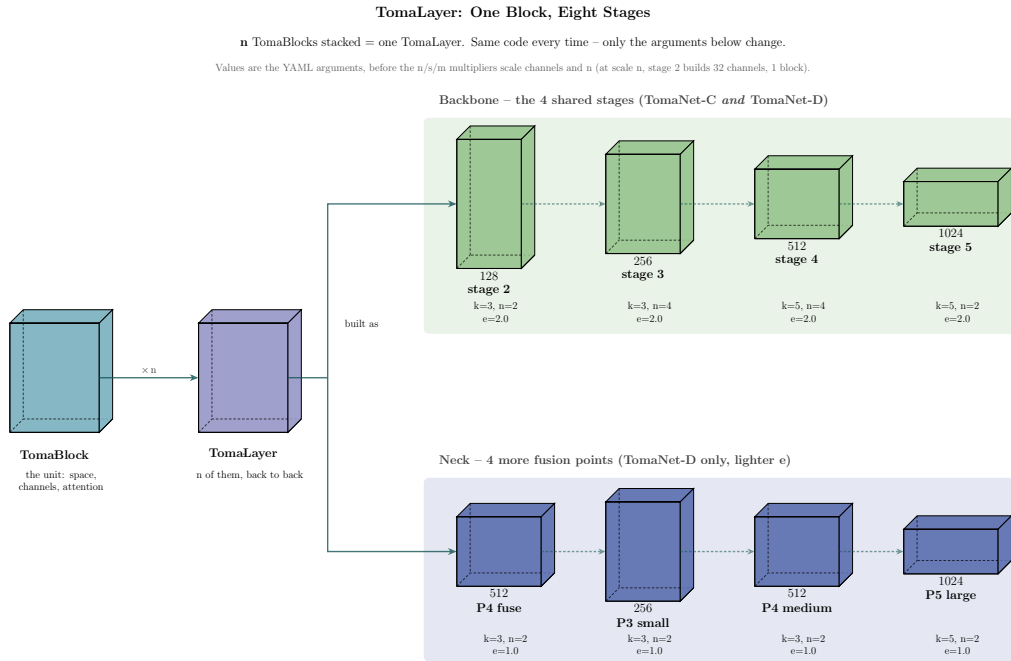


Figure 1: One block, stacked into a **TomaLayer**, then instantiated eight times: four shared backbone stages (green, in both models) and four neck fusion points (blue, TOMANET-D only). Tile height tracks feature-map size and tile width tracks channels, so the shape matches the real models; the values under each tile are the literal YAML arguments.

2 Inside one TomaBlock

A **TomaBlock** runs five steps in a fixed order, then optionally adds the input back. Written in the order they execute (this is literally `TomaBlock.forward` in `tomanet/modules.py`):

1. **RepPConv (partial convolution, 3×3)**. Convolves only the first quarter of the channels and passes the other three quarters straight through, then concatenates. Convolutioning fewer channels means less memory traffic, which is the real bottleneck on small devices — not the arithmetic. At training time this step has three parallel branches (3×3 , 1×1 , identity, each with its own BatchNorm); at deployment they collapse into a single 3×3 convolution.

2. 1×1 **expand**. Widens the channels to `hidden = int(c1 × e)`, giving the next step room to work in.
3. **Depthwise convolution**, $k \times k$. One filter per channel, so a larger k costs almost nothing. This is the *only* place k is used.
4. **CoordAtt (coordinate attention)**. Pools along height and width separately, so the attention weights remember *where* something is, not just which channel it was in. Lesions are small and position-specific, which is why plain channel-only attention (SE/ECA) was not used here.
5. 1×1 **project**. Brings the channels back down to `c2`.
6. **Residual add**, but only when `c1 == c2`. When the block changes the channel count there is nothing to add, so it is skipped automatically.

The bottom row of Figure 2 shows these steps as a chain.

3 The parameters TomaLayer takes

TomaLayer has seven arguments. Two are filled in for you, and in practice only three are ever varied across the whole project.

Argument	What it controls	Default	Used in TomaNet
<code>c1</code>	input channels	—	filled in automatically from the previous layer
<code>c2</code>	output channels	—	128 / 256 / 512 / 1024, then width-scaled
<code>n</code>	how many TomaBlocks are stacked	1	2 or 4, then depth-scaled
<code>k</code>	kernel of the depthwise step	3	3 in early stages, 5 in deep stages
<code>e</code>	hidden width = <code>c1 × e</code>	2.0	2.0 in the backbone, 1.0 in the neck
<code>attn</code>	use CoordAtt (else identity)	True	True everywhere
<code>rep</code>	use the 3-branch RepPConv	True	True everywhere

Two details worth knowing:

- Only the *first* block in a TomaLayer maps `c1`→`c2`; the remaining `n−1` blocks are `c2`→`c2`, so they all get the residual connection.
- The quarter-of-the-channels split is `ratio=0.25` inside TomaBlock. It is not exposed on TomaLayer, because nothing in either model changes it.

In a model YAML one call site is a single line, where the number before the module name is `n`:

```
- [-1, 4, TomaLayer, [256, 3]] # n=4, c2=256, k=3, e defaults to 2.0
- [-1, 2, TomaLayer, [512, 3, 1.0]] # n=2, c2=512, k=3, e=1.0 (neck)
```

4 How the same block plugs into both models

Ultralytics has no plugin registry, so two small things make TomaLayer usable from a YAML:

1. `tomanet.register.register()` binds TomaBlock, TomaLayer and CoordAtt into the namespaces that the YAML parser resolves names against.
2. `scripts/patch_ultralytics.py` adds those names to the parser’s channel-handling sets, so `c1/c2` are injected and the width multiplier is applied. Without the patch a model still builds, but the channels are not scaled — so `verify_setup.py` fails loudly if it is missing.

After that, using the block in either model is just writing the line above into `tomanet-cls.yaml` or `tomanet-det.yaml` under `configs/models/`.

TomaNet-C — classification

Stem (two strided convolutions) → four TomaLayer stages → global average pool → one fully-connected layer. Layers 0–8 are the trunk.

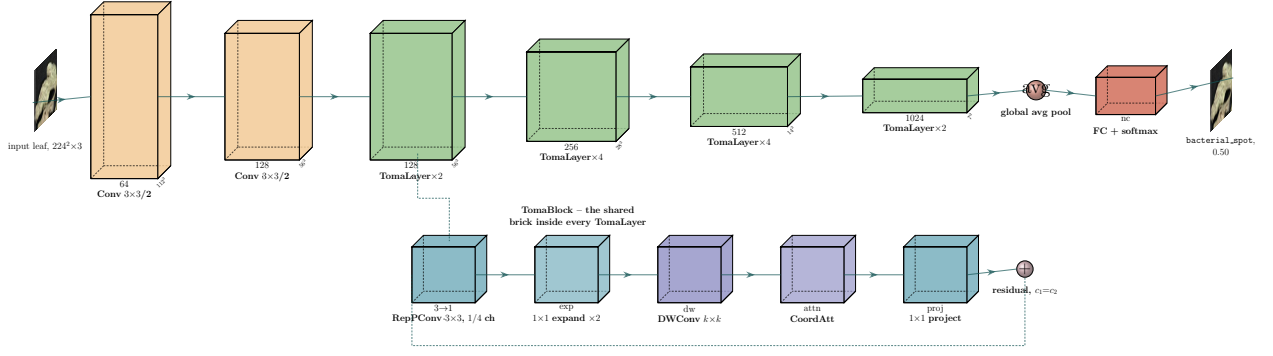


Figure 2: TOMANET-C. The bottom row is the TomaBlock breakdown from Section 2.

TomaNet-D — detection

The same layers 0–8, byte for byte, then an SPPF block and a PAFPN neck that fuses three resolutions, ending in three detect heads — one each for small, medium and large objects.

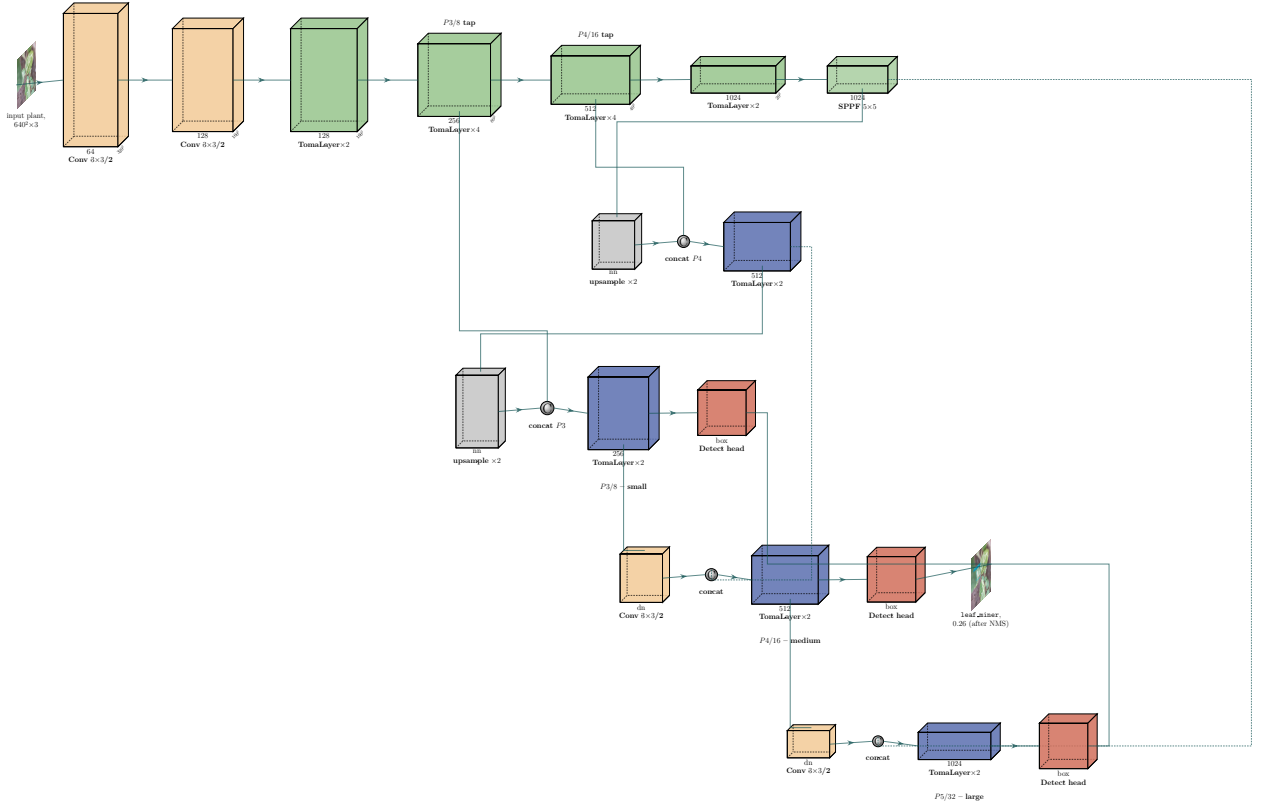


Figure 3: TOMANET-D. Everything green is the shared trunk; everything blue is the neck, which is where the four extra TomaLayer calls live. Full-size version: [figures/fig_tomanet_d.pdf](#).

All eight call sites

Where	Layer	c2	k	n	e	Present in
Backbone stage 2	2	128	3	2	2.0	C and D
Backbone stage 3	4	256	3	4	2.0	C and D
Backbone stage 4	6	512	5	4	2.0	C and D
Backbone stage 5	8	1024	5	2	2.0	C and D
Neck, top-down $P4$	12	512	3	2	1.0	D only
Neck, $P3$ (small)	15	256	3	2	1.0	D only
Neck, $P4$ (medium)	18	512	3	2	1.0	D only
Neck, $P5$ (large)	21	1024	5	2	1.0	D only

These are the YAML arguments, not the built model. The width and depth multipliers are applied on top, so at scale n stage 2 is actually built with 32 channels and a single block, and at scale m with 96 channels and two blocks. The table above is what you edit; Section 5 is what you get.

Because the first nine layers are identical in both YAMLs, a trunk trained on classification can initialise the detector and vice versa. That is a property of the files, not a claim — an automated test compares the two layer lists.

5 Why TomaNet is tunable

Everything below was measured on this machine with `scripts/verify_setup.py` and by rebuilding modified configs; none of it is estimated.

Size: the width and depth multipliers

Each model ships in three sizes. The multipliers scale `c2` (width) and `n` (depth) everywhere at once, so one number changes the whole network.

Scale	depth $\times$	width $\times$	TomaNet-C @ 224px	TomaNet-D @ 640px
n	0.50	0.25	1.32 M / 0.37 GFLOPs	2.92 M / 8.58 GFLOPs
s	0.50	0.50	4.47 M / 1.32 GFLOPs	10.64 M / 29.68 GFLOPs
m	1.00	0.75	14.44 M / 4.56 GFLOPs	30.79 M / 82.84 GFLOPs

For reference, the shared trunk alone at scale n is 0.97 M parameters — about three quarters of the whole classifier. The neck and the three detect heads are what the detector adds on top of it.

Cost: the expansion factor

`e` is the cheapest knob in the model, because it sets the width of the most expensive step. The neck runs at `e=1.0` rather than the default 2.0, which is where most of the detector’s savings come from:

TomaNet-D at scale n	Parameters	GFLOPs @ 640px
neck at <code>e=2.0</code> (the default)	3.60 M	10.32
neck at <code>e=1.0</code> (what ships)	2.92 M	8.58
<i>saved</i>	<i>0.68 M</i>	<i>1.74</i>

The backbone is untouched by this, so the trunk stays byte-identical to TOMANET-C.

Accuracy features that are nearly free

- **CoordAtt** (`attn`). Turning it off everywhere in TOMANET-D-*n* takes the model from 2.92 M to 2.84 M parameters and from 8.58 to 8.57 GFLOPs — roughly 0.08 M parameters and 0.01 GFLOPs for the whole network. It is kept on because the cost is that small.
- **RepPConv** (`rep`). Three branches at training time, one 3×3 convolution after `fuse()`, so inference pays for one branch. The fold is exact to floating-point noise: the largest difference between the unfused and fused outputs is 2.4×10^{-7} .
- **Kernel size** (`k`). 3 in the early, high-resolution stages and 5 in the deep, low-resolution ones. Because the step is depthwise, the larger kernel buys a wider receptive field for almost no extra parameters.

What to edit, in practice

- **Need it smaller or larger?** Change the scale letter (`n/s/m`). Nothing else has to change.
- **Need it cheaper without shrinking the channels?** Lower `e` at specific call sites, as the neck already does.
- **Need more capacity in one place only?** Raise `n` on that one line.
- **Deploying?** Call `fuse_reparam(model)` after loading weights so the three training branches collapse.

The scale-suffixed YAMLs (`tomanet-clsn.yaml`, `tomanet-detn.yaml`, ...) are regenerated from the two base configs on every run, so edits belong in `tomanet-cls.yaml` and `tomanet-det.yaml` only.